

Pertemuan 5

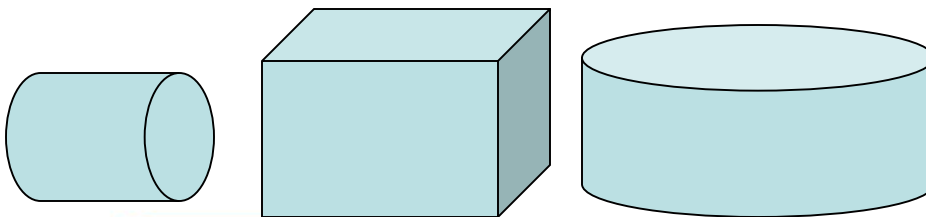
STACK atau TUMPUKAN



STACK (TUMPUKAN)

Merupakan bentuk khusus dari Linier List yang pemasukan dan penghapusan elemennya hanya dapat dilakukan pada satu posisi, yaitu posisi akhir dari List (Top)

Prinsip Stack adalah ***LAST-IN-FIRST-OUT (LIFO)***.



Klik untuk
Ilustrasi Stack

OPERASI STACK

- **EMPTY**
Untuk memeriksa apakah stack kosong
- **SIZE**
mengembalikan nilai stack
- **PUSH**
Untuk menambahkan item pada posisi paling atas
- **POP**
Untuk menghapus item paling atas (TOP)
- **TOP**
mengembalikan posisi ke elemen paling atas
- **NOEL**
menampilkan jumlah elemen pada stack

Inisialisasi

```
stack = []

def push(value):
    stack.append(value)

def pop():
    stack.pop()

def noel():
    print len(stack)

def top():
    top = len(stack) - 1
    if top < 0:
        print "Tidak terdefinisi"
    else:
        print stack[top]

def isempty():
    if len(stack) == 0:
        print "True"
    else:
        print "False"

def tampilkan(stack):
    print stack

while True:
    value = input("--> ")
```

stack = [] ==> Digunakan untuk membuat stack dengan nama "stack" yang masih kosong

def push(value): ==> Digunakan untuk mendefinisikan sebuah fungsi "push" yang diisi dengan nilai

stack.append(value) ==> Digunakan untuk menambahkan elemen pada puncak stack dengan nilai dari fungsi "push"

def pop(): ==> Digunakan untuk mendefinisikan sebuah fungsi "pop"

stack.pop() ==> Digunakan untuk menghapus elemen dari puncak stack dari fungsi "pop"

def noel(): ==> Digunakan untuk mendefinisikan sebuah fungsi "noel"

print len(stack) ==> Digunakan untuk menampilkan jumlah elemen di dalam stack dengan memanggil nama "stack" yang telah dibuat

def top(): ==> Digunakan untuk mendefinisikan sebuah fungsi "top"

top = len(stack) - 1 ==> Sebagai perhitungan untuk mengetahui elemen pada puncak stack

print stack[top] ==> Maka akan menampilkan elemen pada puncak stack

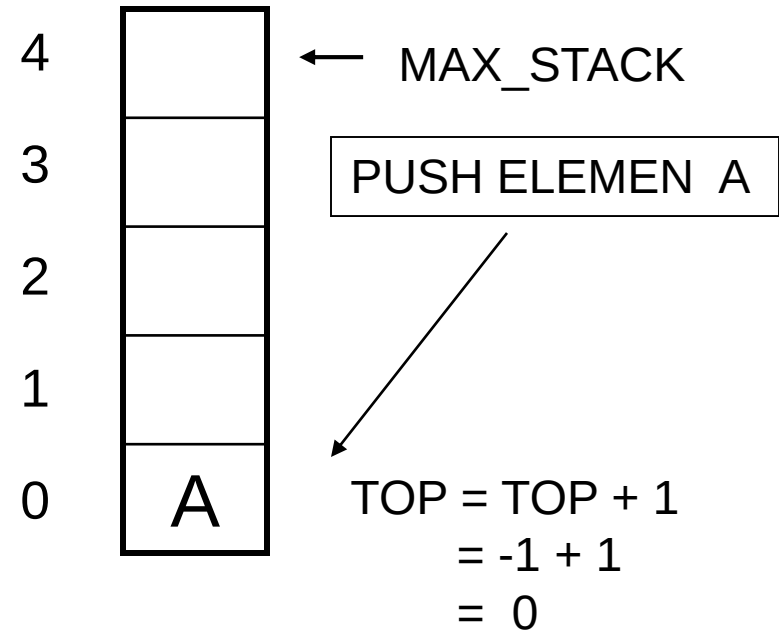
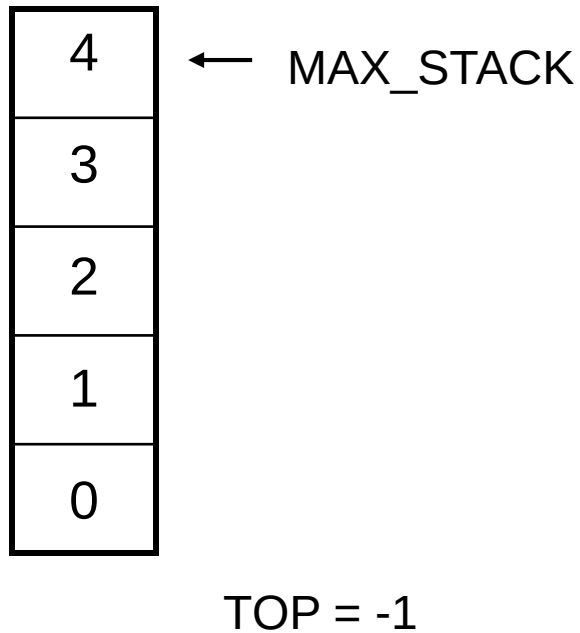
def isempty(): ==> Digunakan untuk mendefinisikan sebuah fungsi "isempty"

def tampilkan(stack): ==> Digunakan untuk mendefinisikan sebuah fungsi "tampilkan"

Fungsi PUSH

- Digunakan untuk memasukkan elemen ke dalam stack dan selalu menjadi elemen teratas stack
- Dengan cara :
 1. Menambah satu (increment) nilai TOP of STACK setiap ada penambahan elemen stack selama stack masih belum penuh
 2. Isikan nilai baru ke stack berdasarkan indeks TOP of STACK setelah ditambah satu (diincrement)

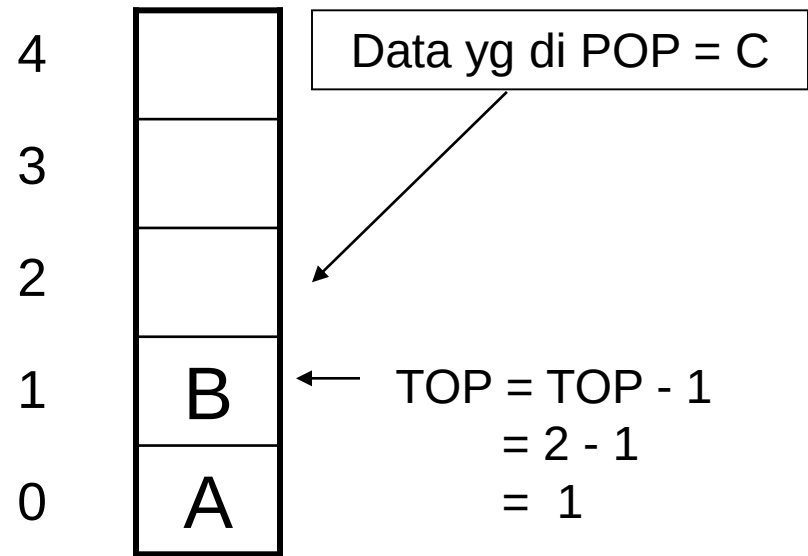
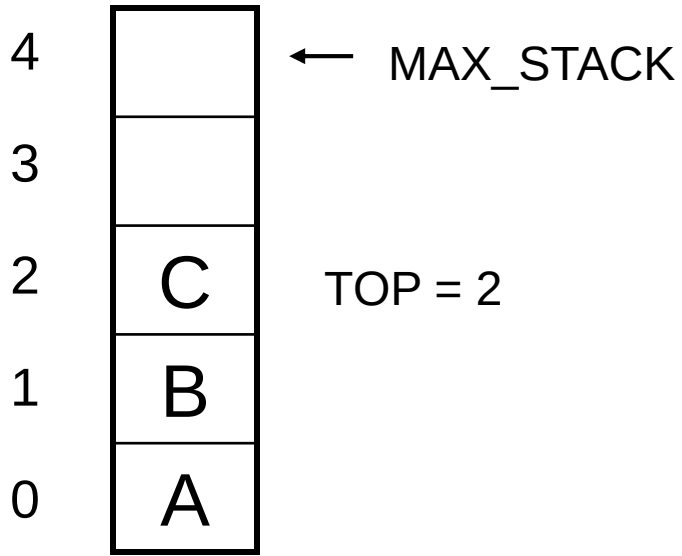
Fungsi PUSH (Lanjutan)



Fungsi POP

- Digunakan untuk menghapus elemen yang berada pada posisi paling atas dari stack.
- Dengan cara :
 1. Ambil dahulu nilai elemen teratas stack dengan mengakses TOP of STACK.
 2. Tampilkan nilai yang akan diambil.
 3. Lakukan decrement nilai TOP of STACK sehingga jumlah elemen stack berkurang 1

Fungsi POP (Lanjutan)



Latihan

Diketahui suatu stack dgn $\text{max_stack} = 6$

1. Bila dilakukan PUSH 3 elemen kedalam stack, kemudian di PUSH lagi 2 elemen dan di POP 3 elemen. Maka dimana posisi Top of Stack ?
2. IsEmpty pada kondisi terakhir adalah ?
3. Dari kondisi diatas, Berapa elemen yg hrs di PUSH unt mencapai kondisi penuh Top of Stack = max_stack ?
4. Berapa elemen yg hrs di POP unt mencapai kondisi Empty = True

```
stack = []
```

```
# append() function to push  
# element in the stack
```

```
stack.append('a')  
stack.append('b')  
stack.append('c')
```

```
print('Initial stack')  
print(stack)
```

```
# pop() function to pop  
# element from stack in  
# LIFO order
```

```
print('\nElements popped from stack:')  
print(stack.pop())  
print(stack.pop())  
print(stack.pop())
```

```
print('\nStack after elements are  
popped:')  
print(stack)
```

```
# uncommenting print(stack.pop())  
# will cause an IndexError  
# as the stack is now empty
```

Output:

```
Initial stack  
['a', 'b', 'c']
```

```
Elements popped from stack:
```

```
c
```

```
b
```

```
a
```

```
Stack after elements are popped:  
[]
```

QUEUE (ANTREAN)



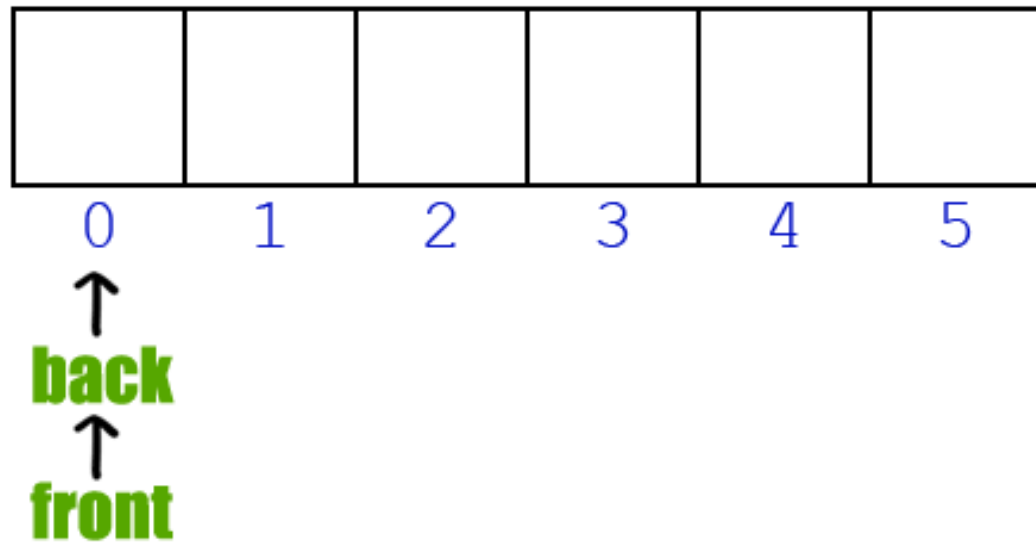
PENGERTIAN QUEUE (ANTREAN)

Struktur Data Antrean (Queue) adalah suatu bentuk khusus dari List Linier dengan operasi pemasukan data hanya diperbolehkan pada salah satu sisi, yang disebut sisi Belakang / ekor (Tail) dan operasi penghapusan hanya diperbolehkan pada sisi lainnya yang disebut sisi Depan / kepala (Head) dari LinkedList.

Prinsip Antrean : FIFO (First In First Out)
FCFS (First Come First Serve)

“Yang Tiba lebih awal Maka akan dilayani Terlebih Dahulu”

Deklarasi Queue



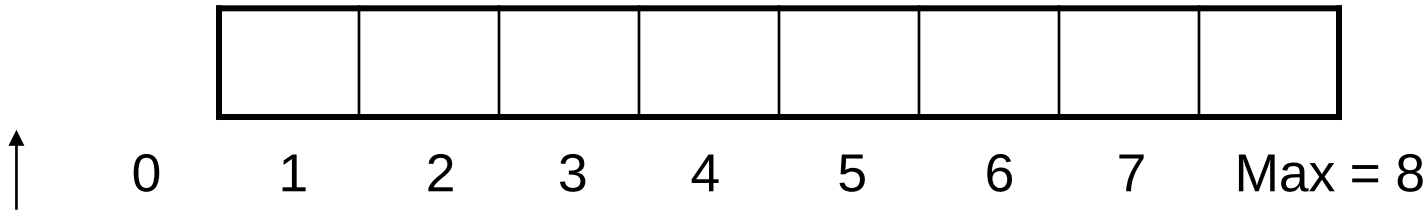
OPERASI QUEUE

- **PEEK ()**
Digunakan untuk mendapatkan elemen yang terdapat pada posisi terakhir tanpa menghapus element tersebut.
- **ISEMPTY()**
Untuk memeriksa apakah queue kosong
- **SIZE()**
mengembalikan jumlah item di dalam list. Tidak memerlukan parameter dan mengembalikan suatu integer.
- **ENQUEUE()**
Untuk menambahkan item pada posisi paling belakang
- **DEQUEUE()**
Untuk menghapus item dari posisi paling depan

Inisialisasi Head

Digunakan untuk membentuk dan menunjukan awal terbentuknya suatu Antrean / Queue

```
def __init__(self):  
    self.head = Node("head")  
    self.size = 0
```



Antrian pertama kali

Fungsi IsEmpty

- Untuk memeriksa apakah Antrian penuh atau kosong
- Head adalah tanda untuk kepala antrian (elemen pertama dalam antrian) yang tidak akan berubah-ubah
- Pergerakan pada Antrian terjadi dengan penambahan elemen Antrian kebelakang,

Fungsi isEmpty (Lanjutan)

```
def isEmpty(self):  
    return self.size == 0
```



↑
0 1 2 3 4 5 6 7 Max = 8

Antrian kosong

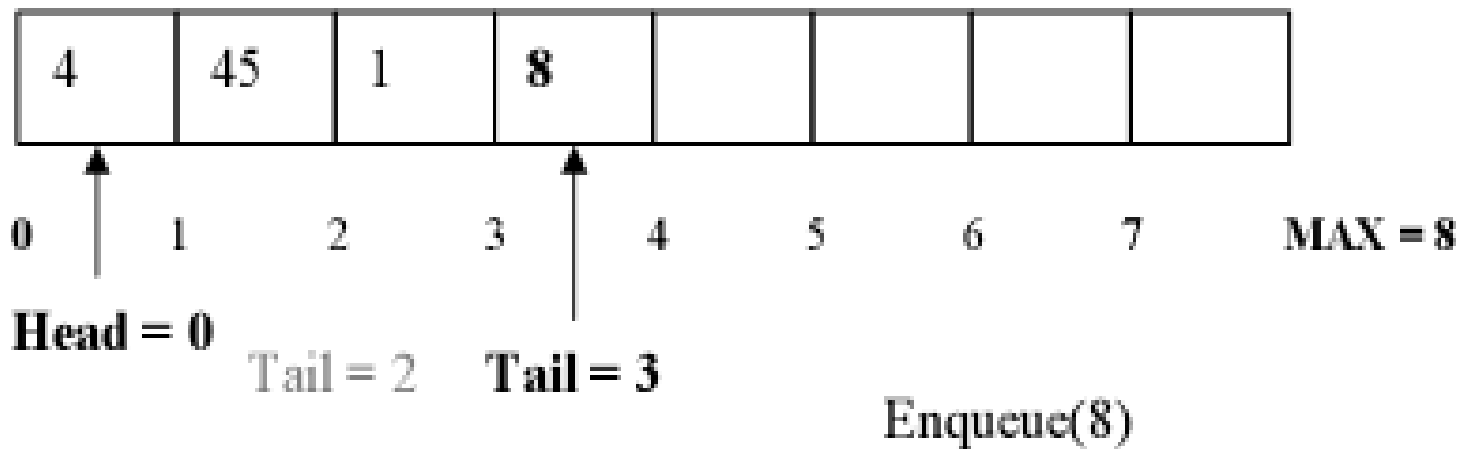
Fungsi Peek

Periksa antrian untuk melihat apakah ada antrian kosong

```
def peek(self):  
  
    if self.isEmpty():  
        raise Exception("Peeking from an empty stack")  
    return self.head.next.value
```

Fungsi Enqueue

- Untuk menambahkan elemen ke dalam Antrian, penambahan elemen selalu dilakukan pada elemen paling **belakang**
- Penambahan elemen selalu menggerakkan variabel Tail dengan cara menambahkan Tail terlebih dahulu



Fungsi Enqueue (Lanjutan)

Menambahkan nilai pada antrian

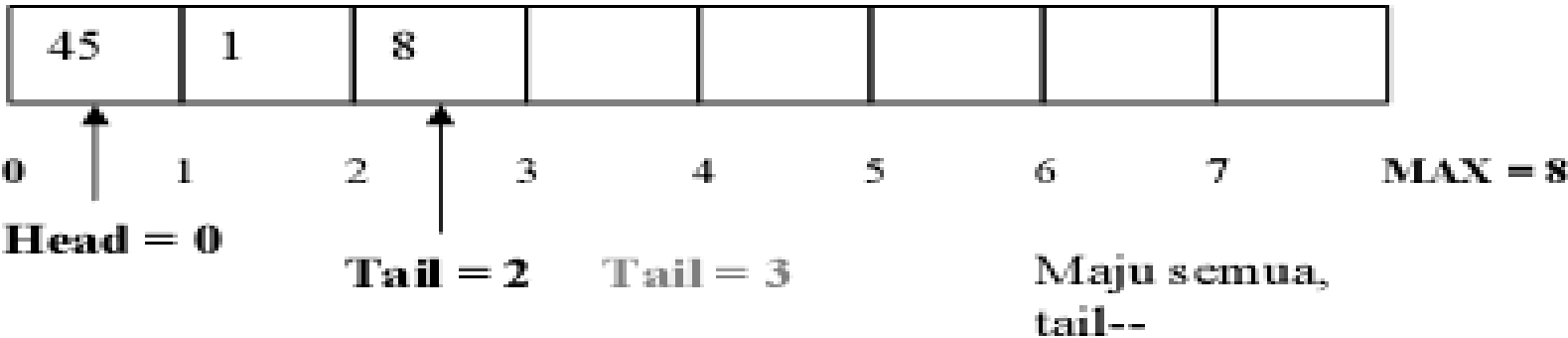
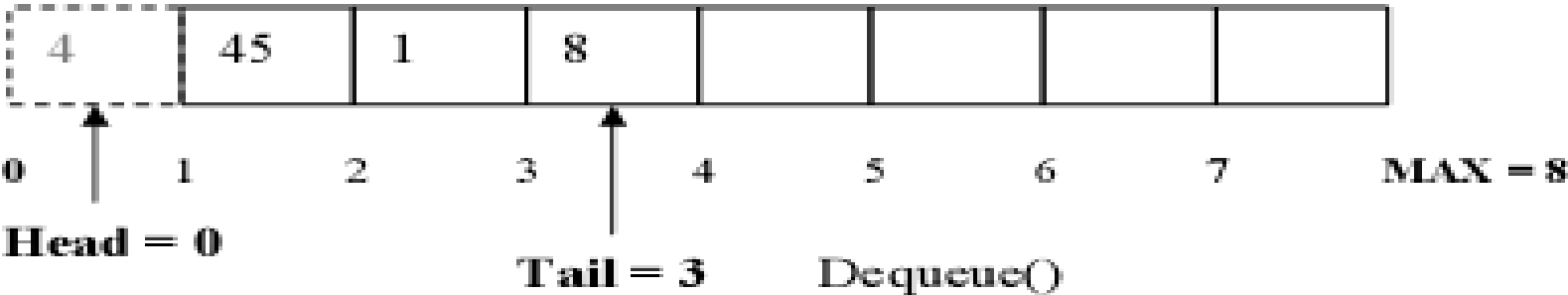
```
def enqueue(self, value):  
    node = Node(value)  
    node.next = self.head.next  
    self.head.next = node  
    self.size += 1
```

Fungsi Dequeue

- Digunakan untuk menghapus elemen terdepan (head) dari Antrian
- Dengan cara : menggeser semua elemen antrian kedepan dan mengurangi Tail dgn 1. Penggeseran dilakukan dengan menggunakan looping

```
def dequeue(self):  
    if self.isEmpty():  
        raise Exception("dequeue from an empty stack")  
    remove = self.head.next  
    self.head.next = self.head.next.next  
    self.size -= 1  
    return remove.value
```

Fungsi Dequeue (Lanjutan)



Fungsi SIZE

digunakan untuk mengetahui banyak elemen atau data yang ada di dalam list.

```
def getSize(self):  
    return self.size
```

Latihan

Berikan gambaran/ilustrasi dari kasus antrian berikut :

- a) Diketahui suatu Antrian/queue dgn $\text{max} = 6$.
- b) Lakukan Enqueue 4 elemen ke dalam antrian, dimanakah posisi Head dan Tail ?
- c) Kemudian lakukan Dequeue 2 elemen dari antrian. Maka dimana posisi Head dan Tail ?
- d) Dari keadaan diatas, bagaimanakah kondisi penuh dan IsEmpty nya ?

```
class Node:
```

```
    def __init__(self, value):  
        self.value = value  
        self.next = None
```

```
class Stack:
```

```
    # Initializing a stack.  
    # Use a dummy node, which is  
    # easier for handling edge cases.
```

```
    def __init__(self):  
        self.head = Node("head")  
        self.size = 0
```

```
    # String representation of the stack  
    def __str__(self):  
        cur = self.head.next  
        out = ""
```

```
        while cur:
```

```
            out += str(cur.value) + "->"  
            cur = cur.next  
        return out[:-3]
```

```
    # Get the current size of the stack
```

```
    def getSize(self):  
        return self.size
```

```
    # Check if the stack is empty
```

```
    def isEmpty(self):  
        return self.size == 0
```

```
    # Get the top item of the stack
```

```
    def peek(self):
```

```
        # Sanitary check to see if we  
        # are peeking an empty stack.
```

```
        if self.isEmpty():  
            raise Exception("Peeking from an  
empty stack")
```

```
return self.head.next.value
```

```
# Push a value into the stack.
```

```
def push(self, value):  
    node = Node(value)  
    node.next = self.head.next  
    self.head.next = node  
    self.size += 1
```

```
# Remove a value from the stack and return.
```

```
def pop(self):  
    if self.isEmpty():  
        raise Exception("Popping from an empty  
stack")  
    remove = self.head.next  
    self.head.next = self.head.next.next  
    self.size -= 1  
    return remove.value
```

```
# Driver Code
```

```
if __name__ == "__main__":  
    stack = Stack()  
    for i in range(1, 11):  
        stack.push(i)  
    print(f"Stack: {stack}")  
  
    for _ in range(1, 6):  
        remove = stack.pop()  
        print(f"Pop: {remove}")  
    print(f"Stack: {stack}")
```